



The Semester Project-IV

File Systems: Formatting a Pen Drive

This article, which is a part of the series on Linux device drivers, takes the next step towards developing a file system module by exploring how to write a formatting application for your real pen drive.

Thanks, friends, for your confidence in Shweta, and not trying to help her figure out the issues with her code. She indeed figured out and fixed the following issues on her own:

- `sfs_read()` and `sfs_write()` need to check for the read and write file permissions before proceeding to read and write, respectively.
- `sfs_write()` should free any previously allocated blocks, as write is always over-write.
- Moreover, the earlier written `sfs_remove()` also now needs to free up the allocated blocks.

SFS format for a real partition

Thereafter, Pugs took the lead and slightly modified Shweta's `format_sfs.c` and `sfs_ds.h` files to format a real pen drive's partition. The key change was that instead of creating the default regular file `.sfs` to format, it would now be operating on an existing block device file corresponding to an underlying partition, say, something like `/dev/sdb1`. So:

- It would get the partition's size from the partition's block device file itself, rather than taking it as a command-line argument.
- Accordingly, it would now expect the partition's block device file name instead of size, as `main()`'s first argument.
- Also, it would not need `mark_data_block()` to grow the file to equal the partition's size.

The `ioctl` command `BLKGETSIZE64` gets the 64-bit size of the underlying block device partition, in bytes. Then, it is divided by the block size (`SIMULA_FS_BLOCK_SIZE`) to get the partition's size in block units. Here is the corresponding modified snippet of the `main()` function in `format_sfs.c`, along with the required `typedef` in `sfs_ds.h`:

```
typedef unsigned long long uint8_t;
...
uint8_t size;

sfs_handle = open(argv[1], O_RDWR);
if (sfs_handle == -1)
{
```

```
    fprintf(stderr, "Error formatting %s: %s\n", argv[1],
    strerror(errno));
    return 2;
}
if (ioctl(sfs_handle, BLKGETSIZE64, &size) == -1)
{
    fprintf(stderr, "Error getting size of %s: %s\n", argv[1],
    strerror(errno));
    return 3;
}
sb.partition_size = size / SIMULA_FS_BLOCK_SIZE;
```

As per the above code, various additional header files would be needed, as follows:

```
#include <errno.h> /* For errno */
#include <string.h> /* For strerror() */
#include <sys/ioctl.h> /* For ioctl() */
#include <linux/fs.h> /* For BLKGETSIZE64 */
```

With all the above changes compiled into `format_sfs`, Pugs plugged in his pen drive, the partition of which was auto-mounted. Then, he took a backup of its contents, and unmounted it, ready for a real SFS formatting of the partition.



Caution: Take a backup of your pen drive's contents—you are formatting it for real. Be careful in choosing the right partition of your pen drive. Otherwise, you may forever lose data from your hard disk, or even make your system unbootable. You have been warned!

Figure 1 demonstrates all of the above, except the backup steps, at the root prompt (`#`). Instead, you may use `sudo`, as well. Note that Pugs got his pen drive partition mounted at `/media/10AC-BF1C`, and the corresponding device file is `/dev/sdb1` (`/dev/sdb` being the complete pen drive). Both these may be different in your case, so take the subsequent steps accordingly. Also, note that the SFS formatting is then started using the following command:

```
# ./format_sfs /dev/sdb1
```